

蓝桥杯30天算法冲刺集训



Day-20 树的直径与DFS序

① Note

本讲义内容可以看做拓展内容，有时间的同学可以学习，时间紧迫的同学可以先暂放。本讲义内容在蓝桥杯国赛里面直接考到的情况较少，但是也并不能说明不会出现，所以还需要进行学习。

定义

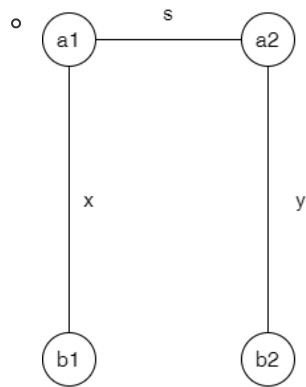
树中两点间的不重复经过的边和点道路称为两点间的路径，路径的长度（路径上所经边的长度和）称为两点的距离。

我们知道圆的直径是一个圆的最长的一条弦。那么，树的直径是树中 两点间的最长路径。通常，我们用一个无序点对 (x, y) 表示一条树的直径。**树的直径可能有很多条。**

性质

树的直径满足如下性质：

1. 若有多条直径，则所有的直径之间必然有公共点。（带有负权边的树除外）
 - 证明：（反证法）如果存在两条直径没有公共点（如下图），因为树必然是连通图，两条直径的任意两个端点间必然存在路径，因此我们一定可以用这两条直径的四个端点中的某两个构造出一条更长的直径。



x, y 为直径, 且不相交, 那么有:
 $s \geq 0$
 所以, $x + s + y \geq x$
 这与 x, y 是直径(最长路径)矛盾

2. 直径的两个端点一定是叶子。

◦ 证明：（反证法）如果不是叶子节点，那么可以利用该节点
 的子节点构造出一条更长的路径（直径）

3. 树中距离某一直径端点最远的点，至少有一个是该直径的另外一个端点。

◦ 证明：（反证法）如果不是，那么我们一定可以用与之距离最
 远的点更新直径。

4. 对树上任意一个点，与之距离最远的每一个点，至少有一个直径的端点。

◦ 设与之距离最远的点为 x ，任取一条直径 (u, v) ，则易证 (u, x)
 或 (v, x) 必定至少有一个是直径。证明如下：（另一种证明方法
 参加下面方法一的证明）

◦ 假设这个点 x 在一条直径 (u, v) 上，易得 u 或 v 中必有一个点
 为距离最远的点，否则与 (u, v) 是直径矛盾

◦ 假设这个点 x 不在任意一条直径上， x 距离某条直径 (u, v) 的
 距离为 s ，且距离它最远的另一个点的距离为 y ，可以得出一个
 结论： $y + s < x/2$ ，所以必有 y 不是最远距离。

求解方法

方法一、两次遍历法

算法步骤：

1. 第一次遍历，找出距离某个结点（比如根结点）最远的一个点 u 。
2. 第二次遍历，找出距离结点 u 最远的一个点 v 。

3. u 到 v 的简单路径，即为树的一条直径， u 和 v 间的距离即为直径的长度。

算法正确性证明如下：

(1) 假设起始点为 a ，先证第一次遍历得到的最远点 u 一定是某一条直径的一个端点，反证法：

任取树的一条直径 (b, c) ，那么直径 (b, c) 和路径 (a, u) 要么没有交点，要么有交点，下面分两种情况讨论：

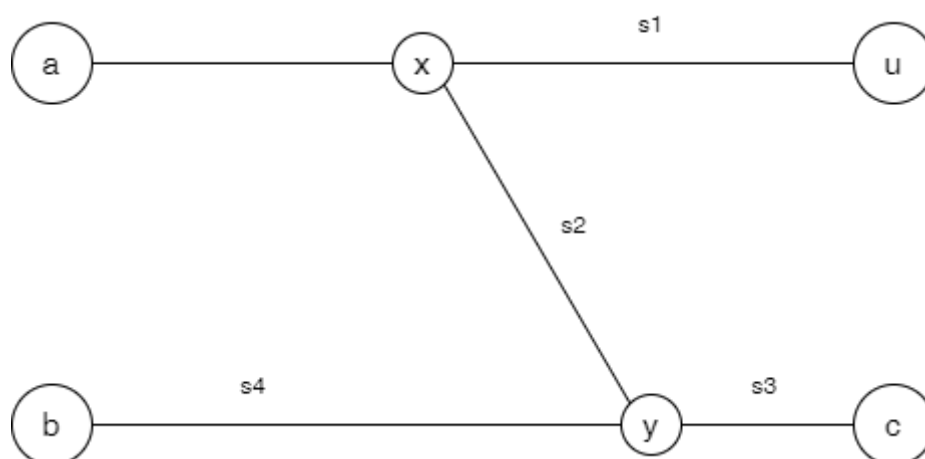
a. 假设不存在交点，则根据树是连通图的性质，路径 (a, u) 上必存在一个节点和直径 (b, c) 上的一个节点连通，我们记为 (x, y) 。

因为 (b, c) 是直径，因此， $s_3 \geq s_1 + s_2$ 。（否则就与 (b, c) 是直径矛盾）

又因为 u 是距离 a 最远的节点，因此 $s_1 \geq s_2 + s_3$ 。

联立上面两个不等式，可以解出： $s_2 = 0$ ，并且 $s_1 = s_3$ 。因此，路径 (b, u) 的长度为 $s_4 + s_1 = s_4 + s_3$ 。

因此， (b, u) 也是一条直径， u 是该条直径的端点。

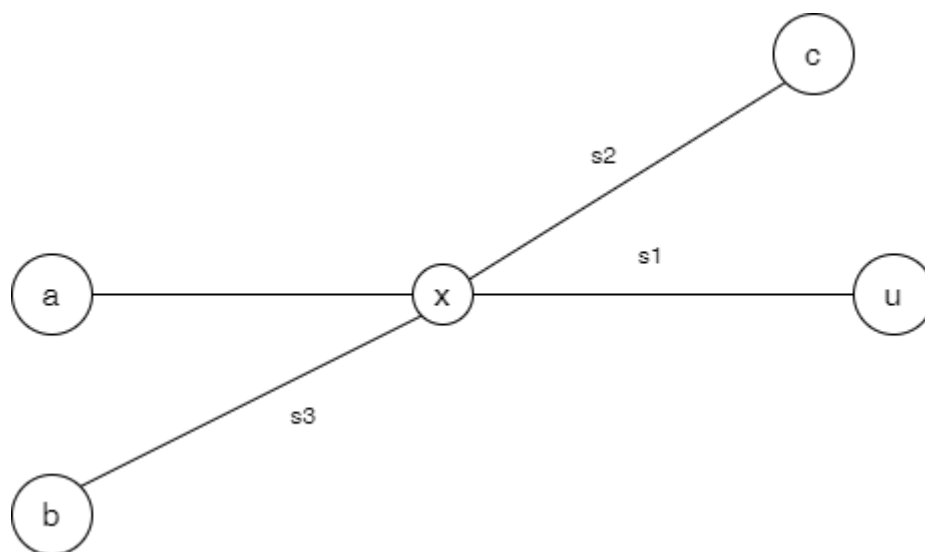


b. 假设存在交点，记该交点为 x ，则有： $s_1 \geq s_2$ ，

又因为 (b, c) 是直径，所以 $s_3 + s_2 \geq s_3 + s_1$ ，

联立上面两个不等式，解得 $s_1 = s_2$ 。

因此， (b, u) 也是一条直径， u 是该条直径的端点。



综上所述可得： u 一定是某一条直径的一个端点。

(2) 第二次遍历，找出距离结点 u 最远的一个点 v 。根据上面的结论， u 是某一条直径的端点，那么 u 距离其最远的点 v 构成的路径 (u, v) 一定是一条直径（因为直径是树上最大的两点间距离），路径 (u, v) 的长度必然也是直径的长度。

时间复杂度

需要对整棵树进行两次遍历，因此时间复杂度为 $O(n)$ 。

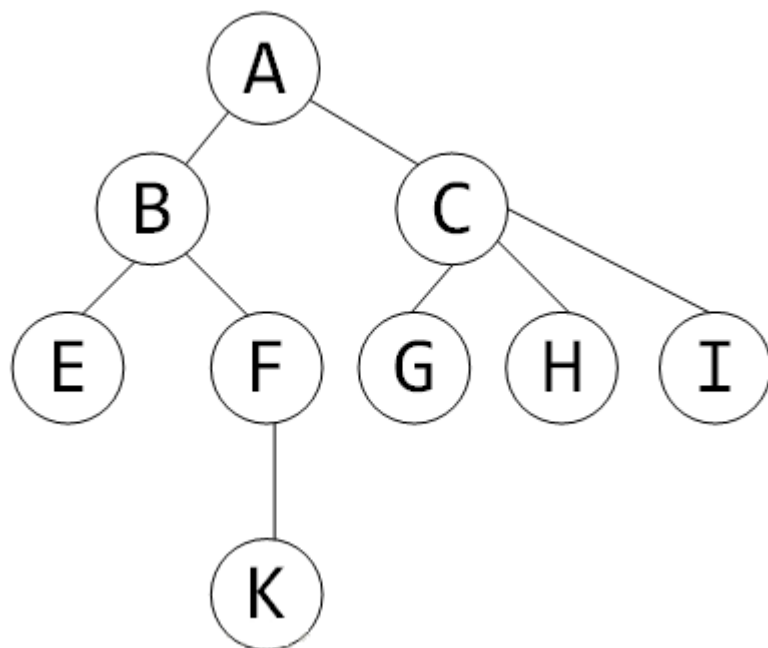
DFS序

定义

DFS 序是指：每个节点在 DFS 深度优先遍历中的进出栈的时间序列。

所以它有什么用呢

我们知道，树是一种非线性的数据结构，它的一些数据调用肯定是没有线性结构来得方便的。所以这个时候，dfs站了出来。基于dfs函数，我们可以在遍历的同时记录下每个节点进出栈的时间序列。比如有这样一颗树：



那么它的 DFS 序列就是：



性质

DFS 序列有一些非常有用的性质：

1. 如果一个点的起始时间和终结时间被另一个点包括，这个点肯定是另一个点的子节点。（简称括号化定理）
2. 一颗子树的所有节点在 DFS 序内是连续的一段；
3. 对于求任意点对 (a, b) 之间的路径的问题，可分为两种情况，首先是令 lca 是 a, b 的最近公共祖先：
 - 若 lca 是 a, b 之一，则 a, b 之间的 in 时刻的区间或者 out 时刻区间就是其路径。例如 AK 之间的路径就对应区间 $ABEEFK$ 或者 $KFBCGGHHIICA$ 。
 - 若 lca 不为 a 或 b ，则 a, b 之间的路径为 $In[a]$ 、 $Out[b]$ 之间的区间或者 $In[b]$ 、 $Out[a]$ 之间的区间。另外，还需额外加上 lca 。比如上图中的 EK 路径，对应为 EFK 再加上 B 。考虑 EH 之间的路径，对应为 $EFKKFBCGGH$ 再加上 A 。

我们发现，一个点的进出栈的时间点之间的时间段就是它的子树在栈中的所有时间。也就是说，子树的 DFS 序肯定大于根节点的进栈时间小于根节点的出栈时间，这就成了一个**区间问题**。所以我们就把一个树上的问题转换到了一个线性的数据结构上面。区间问题就比较好做了，有各种强大的数据结构可以用来维护区间，例如线段树，树状数组。然后我们可以**随便搞了**。

因为我们发现每个节点在 DFS 序中均会出现 2 次，第一次是进入 DFS 的时刻，第二次是离开 DFS 的时刻。分别称之为 *in* 与 *out*。在区间操作中，如果某个节点出现了 2 次，则该节点将会出现被“抵消”。所以通常会将 *out* 时刻对应的点设置为**负数**。那么，**根节点到任意节点的权值和就是 DFS 序上对应的一个前缀和**。

利用这些性质，可以利用 DFS 序列完成子树操作和路径操作，同时也有可能将莫队算法应用到树上从而得到树上莫队。

时间戳

时间戳是 DFS 序的另外一种形式，它记录第节点的一次**开始**访问这个点的时间和和最后**结束**访问的时间。

```
void dfs(int u, int fa) {
    seq[++cnt] = u;    // 记录 dfs 序列
    in[u] = cnt;
    for(auto v : g[u]) {
        if(v == fa) continue;
        dfs(v, u);
    }
    // seq[++cnt] = u;    // 和标准 DFS 序列不同，时间戳不重复保存元素
    out[u] = cnt;
    // siz[u] = out[u] - in[u] + 1;    // 子树元素数量
}
```

蓝桥杯真题

异或和（蓝桥杯Python2023B组省赛）

可以看到是树上单点修改，子树查询。用我们上面说的DFS序的性质，一个节点要是包含在另一个节点的 in 节点和 out 节点之内，那么这个节点就是其子树内的节点。

那么我们直接使用DFS序+线段树即可

```
#include <bits/stdc++.h>
using namespace std;
const int N = 1e5 + 10;
typedef long long ll;
int a[N], in[N], out[N], cnt, bk[N];
vector<int> G[N];
struct node
{
    int l, r, v;
} t[N<<2];
void build(int p, int l, int r)
{
    t[p].l = l; t[p].r = r;
    if(t[p].l == t[p].r)
    {
        t[p].v = a[bk[l]];
        return;
    }
    int mid = (l + r) >> 1;
    build(p << 1, l, mid);
    build(p << 1 | 1, mid + 1, r);
    t[p].v = t[p << 1].v ^ t[p << 1 | 1].v;
}
void change(int p, int l, int r, int z)
{
    if(t[p].l == t[p].r && t[p].l == l)
    {
        t[p].v = z;
    }
}
```

```

        return;
    }
    int mid=(t[p].l+t[p].r) >> 1;
    if(l<=mid) change(p<<1,l,r,z);
    if(r>mid) change(p<<1|1,l,r,z);
    t[p].v = t[p<<1].v ^ t[p<<1|1].v;
}
ll query(int p,int l,int r)
{
    if(t[p].l>=l&& t[p].r<=r) return 1LL*t[p].v;
    int mid = (t[p].l+t[p].r) >> 1;
    ll ans=0;
    if(l<=mid) ans^=query(p<<1,l,r);
    if(r>mid) ans^=query(p<<1|1,l,r);
    return ans;
}
void dfs(int p,int fa)
{
    bk[++cnt]=p;
    in[p]=cnt;
    for(auto son:G[p])
        if(son!=fa) dfs(son,p);
    out[p]=cnt;
}
int main()
{
    int n,m,x,y;
    cin>>n>>m;
    for(int i=1;i<=n;i++) cin>>a[i];
    for(int i=1;i<n;i++)
    {
        cin>>x>>y;
        G[x].push_back(y);
        G[y].push_back(x);
    }
    dfs(1,0);
    build(1,1,n);
    while(m--)
    {

```

```

    int id;
    cin>>id;
    if(id==1)
    {
        cin>>x>>y;
        change(1,in[x],in[x],y);
    }
    else
    {
        cin>>x;
        cout<<query(1,in[x],out[x])<<"\n";
    }
}
}

```

习题

无权树直径

```

#include<bits/stdc++.h>
using namespace std;

const int N = 1e5+5;
int n;
vector<int> g[N];
int dis[N];

int dfs(int u, int fa)    // 利用 fa 避免重复搜索,可以和下面利用
vis[] 的广搜写法对比
{
    int ans = u;        // ans: 距离当前节点最远的节点编号
    for(auto v : g[u])
    {
        if(v == fa) continue;    // 往 fa 方向去搜索是重复搜索,跳
过

        dis[v] = dis[u] + 1;    // 更新当前节点距离
        int d = dfs(v, u);    // 找到从 v 出发的最远点
    }
}

```

```

        if(dis[d] > dis[ans]){
            ans = d;
        }
    }
    return ans;
}

int main()
{
    cin >> n;
    for(int i=1, a, b;i<n;i++){
        cin >> a >> b;
        g[a].push_back(b);    // 注意：这棵树应该看作是无根树，因此，父结点也要存进邻接表中
        g[b].push_back(a);
    }
    int st = dfs(1, 0);
    dis[st] = 0;
    int ed = dfs(st, 0);
    cout << dis[ed] << endl;
    return 0;
}

```

涨工资

思路分析：

子树结点在 DFS 序列上是连续的，于是子树修改映射到 DFS 序列上就变成了区间修改，于是可以用差分去处理。

```

#include <bits/stdc++.h>
using namespace std;

const int N = 1e5+5;
long long n, q, e[N];
long long seq[N], in[N], out[N], w[N], d[N], cnt;
long long ans;
vector<int> g[N];

```

```

void dfs(int u) { // 转 DFS 序
    seq[++cnt] = u;
    w[cnt] = e[u];
    in[u] = cnt;
    for(auto v : g[u]) {
        dfs(v);
    }
    out[u] = cnt;
}

int main()
{
    cin >> n >> q;
    for(int i = 1; i <= n; i++) cin >> e[i];
    for(int i = 1; i < n; i++) {
        int u, v; cin >> u >> v;
        g[u].push_back(v);
    }
    dfs(1);
    for(int i = 1; i <= n+1; i++) { // 转差分
        d[i] = w[i] - w[i-1];
    }
    while(q--) {
        int t, u, k;
        cin >> t >> u >> k;
        if(t == 1) {
            d[in[u]] += k;
            d[in[u]+1] -= k;
        }
        else {
            d[in[u]] += k;
            d[out[u]+1] -= k;
        }
    }
    for(int i = 1; i <= n; i++) {
        d[i] = d[i-1] + d[i];
        ans += d[i];
    }
    cout << ans << '\n';
}

```

```
    for(int i = 1; i <= n; i++) cout << d[in[i]] << ' ';  
    return 0;  
}
```